Fundamental Registers in Basic Computer Architecture

Comprehensive Theory, Bus System Interconnect, Tabled Execution Trace, and Circuit Diagrams

Course: Computer Organization and Architecture | **Topic:** Basic Computer Register Set & Common Bus System

Executive Summary & Architectural Overview

In a basic computer system (such as the classic Mano Machine model), the Central Processing Unit (CPU) relies on a dedicated set of internal hardware registers. Registers are high-speed storage elements constructed from flip-flops that store binary information during program execution. Memory contains instructions and data, but processor registers hold immediate operational operands, instruction codes, memory pointers, and I/O buffer bytes.
The architecture evaluated here comprises an overall memory unit of 4096×16 bits (4K words of 16 bits each). Consequently, memory addresses require **12** bits ($2^{12} = 4096$), while data words require **16** bits. Input and output character transfers utilize **8** bits (ASCII).

1 Detailed Theoretical Analysis of Registers

The basic computer contains eight primary hardware registers, each serving a specialized role in the fetch-decode-execute instruction cycle and input/output subsystem.

Address Register (AR)

Bit Width: 12 Bits | **Bus Selection Code:** $(001)_2$ | **Directly Memory Connected:** **Yes**

The Address Register (AR) holds the 12-bit address of the memory location currently being accessed for a read or write operation. Because memory capacity is 4096 words, AR is 12 bits wide ($2^{12} = 4096$).

- **Function:** Provides physical address signals directly to the address lines of the RAM module.
- **Control Inputs:** Load (LD), Increment (INR), Clear (CLR), Clock (CLK).
- **Microoperations:** $\text{AR} \leftarrow \text{PC}$ (during instruction fetch), $\text{AR} \leftarrow \text{IR}(0-11)$ (during address decoding), $\text{AR} \leftarrow \text{AR} + 1$.

Program Counter (PC)

Bit Width: 12 Bits | **Bus Selection Code:** $(010)_2$ | **Role:** Instruction Sequence Control

The Program Counter (PC) contains the 12-bit address of the next instruction to be fetched from memory.

- **Function:** Maintains the sequential flow of program execution. After fetching an instruction into the Instruction Register, PC is automatically incremented by 1.
- **Branching Logic:** During jump (JMP) or branch instructions, the target address from AR or IR is loaded into PC, altering sequential execution.
- **Microoperations:** $\text{PC} \leftarrow \text{PC} + 1$ (Fetch phase step T_1), $\text{PC} \leftarrow \text{AR}$ (Branching step).

Data Register (DR)

Bit Width: 16 Bits | **Bus Selection Code:** $(011)_2$ | **Role:** Memory Operand Buffer

The Data Register (DR) holds the 16-bit operand read from memory or the operand ready to be written to memory.

- **Function:** Serves as a buffer between high-speed CPU registers/ALU and memory data lines. During arithmetic/logic execution (e.g., `ADD`, `AND`), DR feeds the second operand directly into the Arithmetic Logic Unit (ALU).
- **Microoperations:** DR $\leftarrow M[\text{AR}]$ (Memory Read), DR $\leftarrow$ AC (Memory Write prep), DR $\leftarrow$ DR $+ 1$.

Accumulator (AC)

Bit Width: 16 Bits | **Bus Selection Code:** $(100)_2$ | **Role:** Primary Working & ALU Register

The Accumulator (AC) is the central general-purpose processing register. It stores the primary operand before an operation and receives the final result produced by the ALU.

- **Extended Bit (E):** AC is associated with a 1-bit overflow/carry flip-flop (E).
- **Microoperations:** AC $\leftarrow$ AC $+$ DR (Addition), AC $\leftarrow$ AC $\wedge$ DR (Logical AND), AC $\leftarrow \overline{\text{AC}}$ (Complement), AC $\leftarrow 0$ (Clear), AC $\leftarrow$ shr AC (Shift Right), AC $\leftarrow$ INPR$(0-7)$ (Input transfer).

Instruction Register (IR)

Bit Width: 16 Bits | **Bus Selection Code:** $(101)_2$ | **Role:** Instruction Storage & Decode

The Instruction Register (IR) receives and holds the 16-bit instruction code fetched from memory.

- **Bit Breakdown:**
  - Bit 15 (I): Direct (0) / Indirect (1) Addressing Mode Flag.
  - Bits 14–12 (Opcode): Operation code decoded by the Control Unit (D_0 through D_7).
  - Bits 11–0 (Address): Operand memory address or un-decoded register/IO command bits.
- **Microoperations:** IR $\leftarrow M[\text{AR}]$ (Fetch phase step T_1).

Temporary Register (TR)

Bit Width: 16 Bits | **Bus Selection Code:** $(110)_2$ | **Role:** Internal Scratchpad Storage

The Temporary Register (TR) is used by the control unit to store temporary intermediate data generated during multi-step microoperations (such as saving the return address during interrupt service subroutines).

- **User Accessibility:** Not accessible to programmer instructions; strictly managed by internal control hardware.
- **Microoperations:** TR $\leftarrow$ PC (during interrupt execution cycle), TR $\leftarrow$ TR $+ 1$.

Input Register (INPR)

Bit Width: 8 Bits | **Bus Selection Code:** None (Direct to ALU) | **Role:** Input Character Buffer

The Input Register (INPR) holds an 8-bit alphanumeric character received from an external input device (e.g., serial keyboard).

- **Handshake Flags:** Paired with Input Flag (FGI). When $FGI = 1$, new character is ready. Reading INPR into AC clears FGI to 0.
- **Microoperations:** $\text{AC}(0-7) \leftarrow \text{INPR}, FGI \leftarrow 0$.

Output Register (OUTR)

Bit Width: 8 Bits | **Bus Selection Code:** None (Loaded from AC) | **Role:** Output Character Buffer

The Output Register (OUTR) holds an 8-bit character transmitted from AC to an external output device (e.g., display monitor).

- **Handshake Flags:** Paired with Output Flag (FGO). When $FGO = 1$, output device is ready to accept character. Transferring $\text{AC}(0-7)$ to OUTR clears FGO to 0.
- **Microoperations:** $\text{OUTR} \leftarrow \text{AC}(0-7), FGO \leftarrow 0$.

2 Tabled Specifications and Common Bus Selection Logic

Connecting multiple registers with individual inter-register lines would require thousands of physical connections. To streamline design, basic computers utilize a **16-Bit Common Bus System**.

Table 1: **Master Register Summary Table for Basic Computer System**

Register Symbol	Register Name	Number of Bits	Bus Selection ($S_2S_1S_0$)	Register Function	Control Inputs
AR	Address Register	12	$(001)_2$	Holds memory address	LD, INR, CLR
PC	Program Counter	12	$(010)_2$	Holds address of next instruction	LD, INR, CLR
DR	Data Register	16	$(011)_2$	Holds memory operand	LD, INR, CLR
AC	Accumulator	16	$(100)_2$	Processor register / ALU destination	LD, INR, CLR
IR	Instruction Register	16	$(101)_2$	Holds fetched instruction code	LD
TR	Temporary Register	16	$(110)_2$	Holds temporary internal data	LD, INR, CLR
INPR	Input Register	8	None (Direct to ALU)	Holds input character	None
OUTR	Output Register	8	None (Loaded from AC)	Holds output character	LD
Memory	Main RAM (4K × 16)	16	$(111)_2$	Memory word addressed by AR	Read / Write

Table 2: **16-Bit Common Bus Encoder Selection Logic Table**

S_2	S_1	S_0	Selected Register Output onto Common Bus	Description / Target Routing
0	0	0	None	Bus lines disabled / High Impedance State
0	0	1	Address Register (AR)	Output bits 0–11 placed on Bus lines 0–11
0	1	0	Program Counter (PC)	Output bits 0–11 placed on Bus lines 0–11
0	1	1	Data Register (DR)	Output 16 bits placed on Bus lines 0–15
1	0	0	Accumulator (AC)	Output 16 bits placed on Bus lines 0–15
1	0	1	Instruction Register (IR)	Output 16 bits placed on Bus lines 0–15
1	1	0	Temporary Register (TR)	Output 16 bits placed on Bus lines 0–15
1	1	1	Memory Unit (M[AR])	Read memory word placed on Bus lines 0–15

3 Tabled Step-by-Step Instruction Execution Trace

To understand how registers interact dynamically across clock timing cycles ($T_0, T_1, T_2, \dots$), consider the execution of two consecutive assembly instructions:

1. `LDA 205` (Direct Address: Load memory contents at address `0x205` into Accumulator `AC`).
2. `ADD 206` (Direct Address: Add memory contents at address `0x206` to `AC`).

Assume initial state: PC = 100, $M[100]$ = Opcode for LDA 205, $M[101]$ = Opcode for ADD 206, $M[205]$ = 0x0015, $M[206]$ = 0x002A, AC = 0x0000.

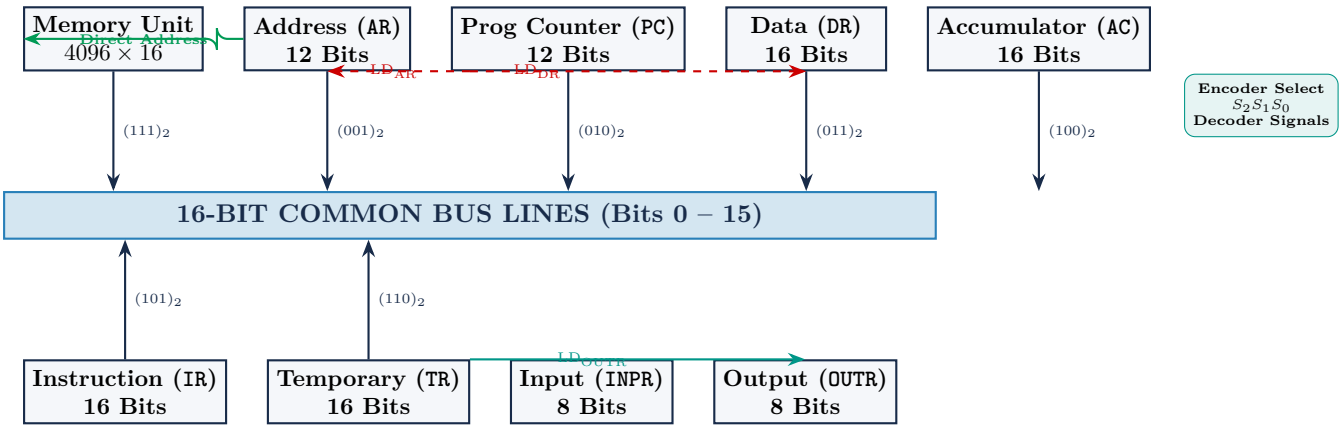
Table 3: **Comprehensive Clock Cycle Register Trace Simulation**

Instruction	Timing	PC	AR	IR	DR	AC	Active Microoperation & Bus Transfer
	T_0	100	100	—-	—-	0000	AR $\leftarrow$ PC (Bus $\leftarrow$ PC, LD_{AR})
	T_1	101	100	LDA 205	—-	0000	IR $\leftarrow M[\text{AR}]$, PC $\leftarrow$ PC + 1
	T_2	101	205	LDA 205	—-	0000	Decode Opcode, AR $\leftarrow$ IR(0 – 11)
	T_3	101	205	LDA 205	—-	0000	Direct Address Mode check ($I = 0$, No-op)
	T_4	101	205	LDA 205	0015	0000	DR $\leftarrow M[\text{AR}]$ (Read Memory)
	T_5	101	205	LDA 205	0015	0015	AC $\leftarrow$ DR, SC $\leftarrow$ 0 (Load complete)
	T_0	101	101	LDA 205	0015	0015	AR $\leftarrow$ PC (Bus $\leftarrow$ PC, LD_{AR})
	T_1	102	101	ADD 206	0015	0015	IR $\leftarrow M[\text{AR}]$, PC $\leftarrow$ PC + 1
	T_2	102	206	ADD 206	0015	0015	Decode Opcode, AR $\leftarrow$ IR(0 – 11)
	T_3	102	206	ADD 206	0015	0015	Direct Address Mode check ($I = 0$, No-op)
	T_4	102	206	ADD 206	002A	0015	DR $\leftarrow M[\text{AR}]$ (Read memory operand)
	T_5	102	206	ADD 206	002A	003F	AC $\leftarrow$ AC + DR, $E \leftarrow$ Cout, SC $\leftarrow$ 0

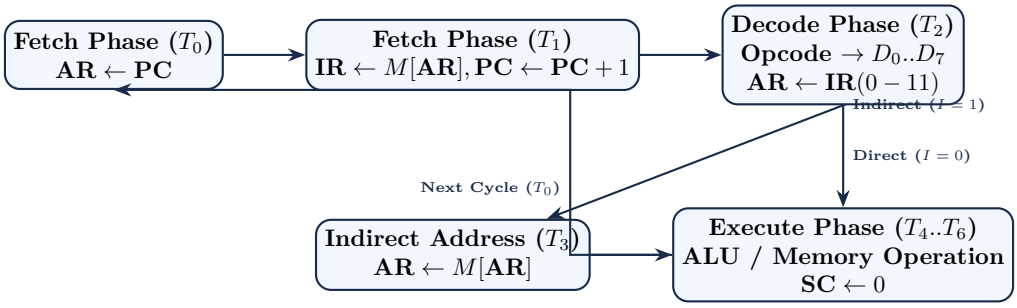
4 Architectural Block Diagrams

4.1 16-Bit Common Bus System Interconnect Diagram

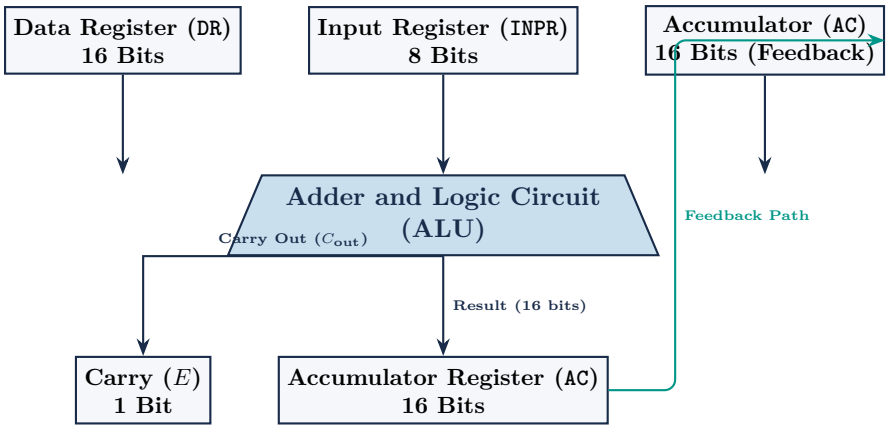
The diagram below shows how registers (`AR`, `PC`, `DR`, `AC`, `IR`, `TR`, `INPR`, `OUTR`) and Memory interact through selector lines S_2, S_1, S_0 and the 16-bit Common Bus.



4.2 Instruction Fetch-Decode-Execute Cycle State Flow



4.3 ALU and Accumulator (AC) Microoperation Logic Block Diagram



Conclusion

The eight basic registers (AR, PC, DR, AC, IR, TR, INPR, OUTR) form the operational backbone of digital computer organization. Coupled with a 16-bit Common Bus system and timing signals generated by the Control Unit decoder, these registers orchestrate data movement, arithmetic computation, and input/output throughput with minimal hardware complexity.